Intune Device Configuration & Policy Troubleshooting Toolkit

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Intune Device Configuration & Policy Troubleshooting (Microsoft Intune) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Intune Device Configuration & Policy Troubleshooting 5

The problem this solves . . . 5

How a policy travels: assignment → scope → sync → CSP apply → report . . . 5

The layered model behind the root-cause tree . . . 5

What "conflict and precedence" means . . . 6

What you get . . . 6

The repeatable method . . . 6

2. Requirements Guide — Intune Device Configuration & Policy Troubleshooting 7

2.1 Intune administrator role (or Policy & Profile manager) . . . 7

2.2 Microsoft Graph read access (for the scripts) . . . 7

2.3 Graph scopes for targeting fixes . . . 7

2.4 Device access for local logs . . . 7

2.5 A reproducing (affected) device and a known-good device . . . 8

2.6 Pilot / diagnostic scope group . . . 8

Checklist . . . 8

Global Requirements — Intune Solutions (category-level) 8

G1. Microsoft Entra ID tenant . . . 8

G2. Microsoft Intune subscription . . . 9

G3. Administrative roles . . . 9

G4. Device enrollment in Intune . . . 9

G5. Pilot security group . . . 9

G6. Supported Windows baseline . . . 9

G7. Network access to Microsoft cloud endpoints . . . 9

3. Diagnostic Playbook — Intune Device Configuration & Policy Troubleshooting 9

Step 0 — Read the report first (always start here) . . . 10

Branch A — "Not assigned" (device never targeted) . . . 10

Branch B — "Assigned but not applied" (targeted, setting still absent) . . . 11

Branch C — Conflict / precedence (setting flips or reports "conflict") . . . 11

Branch D — Filter excludes the device . . . 12

Branch E — Stuck sync / device not checking in . . . 12
Branch F — CSP error code on a setting . . . 12
Where each branch lands . . . 13
5. Testing Guide — Intune Device Configuration & Policy Troubleshooting 13
Test 1 — Assignment reaches the device (Branch A) . . . 14
Test 2 — Filter includes the device (Branch D) . . . 14
Test 3 — Device has checked in (Branch E) . . . 14
Test 4 — Setting applied on the device (Branch B) . . . 14
Test 5 — No conflict on the setting (Branch C) . . . 14
Test 6 — CSP error cleared (Branch F) . . . 14
Test 7 — Known-good device still healthy (regression) . . . 14
Pass/fail gate . . . 15
6. Troubleshooting Guide — Intune Device Configuration & Policy Troubleshooting 15
A. Dynamic group membership lag . . . 15
B. User-context vs device-context mismatch . . . 15
C. "Succeeded" in Intune but wrong value on the device . . . 15
D. Filter evaluates differently than expected . . . 16
E. Settings-catalog vs template/ADMX behaviour . . . 16
F. Sync succeeds but setting never appears . . . 16
G. Diagnostics quick reference . . . 16
H. When to escalate . . . 16
7. Best Practices — Intune Device Configuration & Policy Troubleshooting 17
Naming (so scope is obvious at a glance) . . . 17
Filters (so scope is predictable) . . . 17
Conflict-free design (so settings don't fight) . . . 17
Operating discipline (so diagnosis stays fast) . . . 18
8. Deliverables Summary — Intune Device Configuration & Policy Troubleshooting 18
Documentation . . . 18
PowerShell scripts . . . 18
What the buyer receives . . . 19
Outcome . . . 19
Appendix A — PowerShell Scripts 20

Executive Summary — Intune Device Configuration & Policy Troubleshooting (Microsoft Intune)

White-label · Intune Solutions

A configuration profile shows **"assigned"** — but the setting never reaches the device, and the admin is left guessing and re-pushing blindly. This toolkit replaces the guessing with a **structured root-cause method**. Every Intune device-configuration and settings-catalog policy travels the same path — **assignment** → **group/filter scope** → **MDM sync** → **CSP apply** → **report** — so diagnosis becomes walking those five layers in order and isolating the **one** that fails.

Written for a competent-but-novice Intune admin, it turns "it isn't working" into a single isolated cause — assignment, filter, conflict, sync, or CSP error code — in minutes instead of hours of trial-and-error.

What you get

- A plain-language **Solution Overview** of the five layers a policy must pass through.
- A **Requirements Guide** with Description / Why / Verify / Implement-if-missing for every

prerequisite (Intune roles, Graph read access, device log access).

- A **Diagnostic Playbook** — a symptom-driven decision tree (not assigned; assigned-not-applied;

conflict/precedence; filter excludes device; stuck sync; CSP error code), each branch with exact checks and the layer it isolates.

- Five **PowerShell scripts** for policy/assignment discovery, per-device applied-state validation,

an assignment + conflict coverage report, safe re-sync remediation, and a pilot target-group builder.

- **Testing** and **Troubleshooting** guides mapping each pass/fail gate to the fix, plus

Best Practices for naming, filters, and conflict-free design.

What this delivers

A repeatable, layered method — backed by read-only diagnostic scripts and `-WhatIf`-safe fix scripts — that isolates why any device-configuration or settings-catalog policy isn't applying, so you fix the actual cause instead of re-pushing and hoping.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

The failing layer, identified — assignment, filter, conflict, sync, or CSP — with a method and scripts you keep and reuse, by a single administrator.

1. Solution Overview — Intune Device Configuration & Policy Troubleshooting

The problem this solves

A configuration profile or settings-catalog policy shows **Assigned** in Intune, yet the setting never reaches the device — and the admin is left re-pushing blindly, hoping something changes. This toolkit replaces guessing with a **structured root-cause method**: you follow the policy along the same path it actually travels, isolate the one layer where it breaks, and apply the typical fix for that layer.

How a policy travels: assignment → scope → sync → CSP apply → report

Every Intune device-configuration or settings-catalog policy must pass through **five layers** to take effect. If any layer fails, the setting never lands — so diagnosis means walking the layers in order and finding the first one that fails.

1. **Assignment (targeting).** The policy is assigned to one or more **Entra security groups**

(Included / Excluded). If the device (or its user) isn't in an *Included* group — or is in an *Excluded* group — the policy is never sent. **Device-context** policies target **device** groups; **user-context** policies target **user** groups. Mixing these up is a top cause of "assigned but nothing happens."

1. **Scope (groups + filters).** Even when a group matches, an **assignment filter** can *include*

or *exclude* the device based on device properties (OS, model, ownership, enrollment). A filter in **Exclude** mode that unexpectedly matches the device silently removes it from scope.

1. **Sync (MDM check-in).** The device must **check in** with the Intune service to receive the

policy. Devices sync on a schedule (roughly every 8 hours, sooner after enrollment) or on demand. A device that hasn't synced — or is offline, or whose MDM enrollment is unhealthy — won't have the policy yet, no matter how correct the assignment is.

1. **CSP apply.** Intune delivers each setting to the Windows **Configuration Service Provider (CSP)**

over the MDM channel. The CSP writes the setting and returns a **status / error code**. If a value is unsupported on that OS build, conflicts with another policy, or the CSP rejects it, the setting fails here even though it was delivered — visible as a per-setting **error code** (e.g. `0x87d1fde8`).

1. **Report.** The device reports success/error back to Intune, surfaced under the policy's

Device status / **Per-setting status**. This is the mirror of layers 1–4: reading it tells you *which* layer failed, so you rarely need to guess.

The layered model behind the root-cause tree

The **Diagnostic Playbook** (document 3) is built directly on those five layers. Each symptom an admin sees maps to the layer that produces it:

Symptom the admin sees	Failing layer	What to check
Policy shows no assigned devices / device not listed	**Assignment**	Group membership, Include vs Exclude, device vs user context
Assigned, device listed, but setting absent	**Scope (filter)** or **Sync**	Filter evaluation; last check-in time
Two policies fight; setting flips or shows "conflict"	**CSP apply (conflict/precedence)**	Duplicate settings across policies
"Error" on one setting with a code	**CSP apply**	Per-setting status + error code meaning
Device never checks in / stale status	**Sync**	MDM enrollment health, network, sync trigger

Because the layers are **ordered**, you always diagnose from the top down: confirm assignment before blaming sync, confirm sync before blaming the CSP. That ordering is what turns a vague "it isn't working" into a single isolated cause.

What "conflict and precedence" means

When **two or more** assigned policies set the **same** setting to **different** values, Intune reports a **conflict** and the setting's effective value is **undefined/last-writer** rather than what you intended. There is no simple "priority number" for configuration profiles — the reliable fix is **conflict-free design**: each setting is owned by exactly one policy. The toolkit's report script surfaces overlapping settings so you can eliminate the conflict at the source.

What you get

- A plain-language model of the five layers (this document).
- A **Requirements Guide** for what you need to diagnose (roles, Graph read access, device log access).
- A **Diagnostic Playbook** — the symptom-driven decision tree with exact checks per branch.
- **Five PowerShell scripts**: discovery, per-device validation, assignment + conflict coverage report, safe re-sync/remediation, and a pilot target-group builder.
- **Testing** and **Troubleshooting** guides plus **Best Practices** for naming, filters, and conflict-free policy design.

The repeatable method

> ***Trace the policy along its five layers → isolate the first failing layer → apply that layer's typical fix → re-check the report.*** *The repeatable pattern behind every branch:* ***Assess → Configure → Pilot → Validate → Expand.***

2. Requirements Guide — Intune Device Configuration & Policy Troubleshooting

Confirm **every** requirement before you diagnose — a missing prerequisite is itself a common cause of "the policy isn't applying." Each entry follows **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

> ***Naming convention (used verbatim throughout this toolkit).*** *Create and reuse these exact object names so the scripts and steps line up: - Pilot device group:* `INTCFG-Pilot-Devices` *- Pilot user group:* `INTCFG-Pilot-Users` *- Diagnostic assignment filter:* `INTCFG-Filter-Windows-Corporate`

2.1 Intune administrator role (or Policy & Profile manager)

Description: An Entra/Intune role able to **read and edit** device configuration policies and their assignments — **Intune Administrator**, or the built-in **Policy and Profile Manager** role. **Why required:** You must be able to open every assigned policy, read its assignments/filters, and (for fixes) edit targeting. Read-only roles can diagnose but not remediate. **How to verify:** `https://entra.microsoft.com` → **Roles and administrators** → confirm your account holds **Intune Administrator** (or **Policy and Profile Manager**). **How to implement if missing:** Entra → **Roles and administrators** → assign **Intune Administrator**. Reserve Global Administrator for tenant-wide changes only.

2.2 Microsoft Graph read access (for the scripts)

Description: Delegated Microsoft Graph read scopes so the discovery/validation/report scripts can enumerate policies, assignments, filters, groups, and per-device state. **Why required:** The read-only scripts call Graph to build the assignment + conflict picture the console shows one policy at a time. Without these scopes the scripts cannot sign in. **How to verify:** Run `INTCFG-Discovery.ps1` — on first run it prompts for consent to `DeviceManagementConfiguration.Read.All`. Successful sign-in confirms access. **How to implement if missing:** Sign in as an admin who can consent, or have a Global Administrator grant admin consent for `DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`, and `Group.Read.All`. Install the SDK with `Install-Module Microsoft.Graph -Scope CurrentUser`.

2.3 Graph scopes for targeting fixes

Description: Write scopes needed **only** when you create/populate pilot groups with `INTCFG-Set-ScopeTargeting.ps1` (`Group.ReadWrite.All`). **Why required:** Building a scoped pilot group writes to Entra; the read scripts never need this. **How to verify:** The script previews actions under `-WhatIf` (its default) without needing write consent; the write scope is requested only when you re-run with `-WhatIf:$false`. **How to implement if missing:** Have a Global Administrator grant admin consent for `Group.ReadWrite.All` before you run the builder without `-WhatIf`.

2.4 Device access for local logs

Description: Administrative (or remote-help) access to at least one **affected device** to read the local MDM diagnostic logs. **Why required:** When the report says a setting errored at the **CSP apply** layer, the

device's local MDM logs carry the CSP node and error detail that the console does not. **How to verify:** On an affected device you can open **Settings** → **Accounts** → **Access work or school** → **Info** → **Create report**, or run `INTCFG-Validation.ps1` locally. **How to implement if missing:** Arrange local admin, Intune **Remote Help**, or a remote session to one representative device in the failing group.

2.5 A reproducing (affected) device and a known-good device

Description: One device that shows the problem **and** one comparable device where the same policy applies correctly. **Why required:** Comparing applied state between the two isolates whether the cause is the *policy* (fails everywhere) or the *device/scope* (fails only on some). **How to verify:** Intune → policy → **Device status** shows at least one **Error/Not applicable** device and at least one **Succeeded** device. **How to implement if missing:** Assign the policy to the `INTCFG-Pilot-Devices` group (see 2.6) containing both a failing and a healthy device.

2.6 Pilot / diagnostic scope group

Description: An Entra security group used to reproduce and test targeting safely, named `INTCFG-Pilot-Devices` (device-context) and/or `INTCFG-Pilot-Users` (user-context). **Why required:** You test targeting and filter changes against a small, named scope before touching production assignments. **How to verify:** Entra → **Groups** → the group(s) exist; or run `INTCFG-Set-ScopeTargeting.ps1` (default `-WhatIf`) to preview creation. **How to implement if missing:** Entra → **Groups** → **New group** → **Security** → name it exactly as above and add members, **or** run `INTCFG-Set-ScopeTargeting.ps1 -WhatIf:$false` to build and populate it.

Checklist

- Intune Administrator / Policy & Profile Manager role — Entra Roles
- Graph read scopes consented — `INTCFG-Discovery.ps1` signs in
- Graph write scope available for targeting fixes (`Group.ReadWrite.All`)
- Local device access for MDM diagnostic logs
- One affected device **and** one known-good device identified
- `INTCFG-Pilot-Devices` / `INTCFG-Pilot-Users` group exists
- Naming convention adopted (groups + filter named exactly as above)

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an

administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Diagnostic Playbook — Intune Device Configuration & Policy Troubleshooting

This is the core of the toolkit: a **symptom-driven decision tree**. Start at **Step 0**, pick the branch that matches what you see, and follow it. Each branch names the **layer it isolates**, the **exact checks**, and the **typical fix**. Work **top-down** — confirm assignment before sync, sync before CSP — because the layers are ordered (see document 1).

All console paths are in the **Microsoft Intune admin center** (`https://intune.microsoft.com`) unless stated. Use the exact object names from document 2 (`INTCFG-Pilot-Devices`, `INTCFG-Pilot-Users`, `INTCFG-Filter-Windows-Corporate`).

Step 0 — Read the report first (always start here)

1. Go to **Devices** → **Configuration** (for settings-catalog/config profiles) → open the affected

policy → **Device status** (and **Per-setting status**). *(For endpoint-security policies: **Endpoint security** → **[the area]** → policy → **Device status**.)*

1. Note the device's line:

• **Not applicable / device absent** → the policy never targeted it → go to **Branch A (Not assigned)**.

• **Pending** for a long time, or "last check-in" is old → go to **Branch E (Stuck sync)**.

• **Error** with a per-setting **code** → go to **Branch F (CSP error code)**.

• **Conflict** → go to **Branch C (Conflict / precedence)**.

• **Succeeded** in Intune but the setting is wrong on the device → go to **Branch B (Assigned, not applied)**.

1. For the tenant-wide picture, run `INTCFG-Discovery.ps1` (policies + assignments + filters) and

`INTCFG-Report.ps1` (assignment + conflict coverage) — these show in seconds what the console shows one policy at a time.

Branch A — "Not assigned" (device never targeted)

Isolates: the **Assignment** layer.

Checks (in order):

1. **Is the device (or its user) in an Included group?** Policy → **Properties** → **Assignments**. Note

each **Included** and **Excluded** group. Run `INTCFG-Discovery.ps1` to list assignments per policy including group IDs.

1. **Confirm membership.** Entra → **Groups** → the Included group → **Members**. For dynamic groups,

confirm the device/user actually matches the membership rule (dynamic evaluation can lag).

1. **Context mismatch (very common):** Is this a **device-context** setting assigned to a **user**

group (or vice-versa)? A device-scoped configuration assigned only to a user group won't apply in device context. Check whether the policy/setting is user- or device-scoped.

1. **Excluded wins:** Is the device in **any Excluded** group? **Exclude always beats Include.**

Typical fix: Add the device/user to the correct **Included** group (or fix the dynamic rule); remove an unintended **Exclude**; re-assign to a **device** group for device-context settings. Use `INTCFG-Set-ScopeTargeting.ps1` (default `-WhatIf`) to build/populate `INTCFG-Pilot-Devices` cleanly, then assign the policy to it. Re-check **Device status** after the next sync.

Branch B — "Assigned but not applied" (targeted, setting still absent)

Isolates: narrows to **Scope (filter)**, **Sync**, or **CSP apply** — resolve in that order.

Checks (in order):

1. **Filter?** Policy → **Assignments** → the assignment row → is a **filter** attached, and is its

mode **Include** or **Exclude**? If a filter is present, jump to **Branch D** to evaluate it, then return.

1. **Has the device actually synced since assignment?** Intune → **Devices** → **All devices** → device →

check **Last check-in**. Or run `INTCFG-Validation.ps1` on the device. If it hasn't checked in since you assigned the policy, jump to **Branch E**, then return.

1. **Did the setting reach the CSP?** Policy → **Per-setting status** for that device. If the setting

shows **Error** with a code, go to **Branch F**. If it shows **Succeeded** but the observed value on the device is still wrong, suspect a **conflict** — go to **Branch C**.

1. **Applicability:** Confirm the setting/CSP is **supported on that OS edition/build** (e.g. some

settings are Enterprise/Education only). An unsupported setting reports **Not applicable**, not an error.

Typical fix: Remove/correct an excluding filter (Branch D); force a check-in (Branch E); resolve a conflict (Branch C) or decode the error (Branch F); or accept **Not applicable** and target a supported edition.

Branch C — Conflict / precedence (setting flips or reports "conflict")

Isolates: the **CSP apply** layer — two policies set the **same** setting differently.

Checks:

1. Policy → **Per-setting status** → the setting shows **Conflict**, or the effective value on the

device doesn't match any single policy.

1. Run `INTCFG-Report.ps1` — it lists settings that appear in **more than one** assigned policy

(overlap = conflict risk) and which groups each targets.

1. Identify **every** assigned policy that contains that setting. Remember: for configuration

profiles there is **no reliable priority order** — a conflicting setting becomes **undefined/last-writer**, not "the strictest wins."

Typical fix (conflict-free design): Make **exactly one** policy own the setting. Remove the duplicate setting from all other assigned policies (or scope them to non-overlapping groups). Re-run `INTCFG-Report.ps1` until the overlap is gone, then re-check **Per-setting status**.

Branch D — Filter excludes the device

Isolates: the **Scope (filter)** layer.

Checks:

1. Policy → **Assignments** → the assignment row → open the attached **filter** and note its **rule** and **mode** (Include/Exclude).
1. **Evaluate the rule against the device.** Devices → **Filters** → the filter → **Preview devices** (Evaluate) — does your affected device appear in the matched set?

• Filter mode **Include**, device **not** matched → device is filtered *out* (excluded).

• Filter mode **Exclude**, device **is** matched → device is filtered *out*.

1. Check the rule properties used (e.g. `operatingSystemSKU`, `deviceOwnership`, `model`, `enrollmentProfileName`) against the device's actual properties in **Devices** → **All devices** → device → **Hardware**. A common trap: an ownership/OS value that doesn't match what you assumed.
1. Run `INTCFG-Discovery.ps1` to list which filter is bound to which assignment and its mode.

Typical fix: Correct the filter rule so the intended devices match, or flip the mode, or detach the filter for the pilot. Standardize on the named filter `INTCFG-Filter-Windows-Corporate` so its rule is known and reused. Re-preview until the affected device is *in* scope, then re-sync.

Branch E — Stuck sync / device not checking in

Isolates: the **Sync (MDM check-in)** layer.

Checks:

1. Intune → **Devices** → **All devices** → device → **Last check-in** time. If it's stale (many hours/ days) the policy simply hasn't arrived yet.
1. Run `INTCFG-Validation.ps1` on the device — it reports MDM enrollment health and the last sync.
2. Confirm the device is **online** and can reach Microsoft endpoints (see global requirement G7).
3. Confirm MDM enrollment is healthy (not dual-enrolled/expired): the device shows as **managed by Intune / MDM** and not in an error state.

Typical fix: Force a check-in — on the device, **Settings** → **Accounts** → **Access work or school** → **Info** → **Sync**, or run `INTCFG-Remediation.ps1` (default `-WhatIf`; re-run with `-WhatIf:$false` to trigger the sync). If enrollment is unhealthy, repair enrollment. Re-check **Device status** after the sync completes.

Branch F — CSP error code on a setting

Isolates: the **CSP apply** layer — the setting was delivered but the CSP rejected it.

Checks:

1. Policy → **Per-setting status** → the failing setting → note the **error code** (e.g. `0x87d1fde8`, `0x87d101f4`).

1. Common meanings:

• `0x87d1fde8` **(SyncML(500) / conflict or internal)** — frequently a **conflict** with another policy → route to **Branch C**.

• `0x87d101f4` **(SyncML(404) / node not found)** — the CSP node/setting **isn't supported** on that OS build/edition → confirm applicability (Branch B, step 4).

• `0x82b00006` **/ generic "Remediation failed"** — the value is invalid for the setting or a prerequisite is missing.

1. Get the CSP detail from the **device's local MDM logs** (requirement 2.4): create the MDM report (**Access work or school** → **Info** → **Create report**) or run `INTCFG-Validation.ps1` to surface the CSP node + status.

Typical fix: For a conflict code, resolve via **Branch C**. For "not supported," target a supported edition/build or drop the setting for that device set. For an invalid value, correct the setting value in the owning policy and re-sync.

Where each branch lands

Branch	Symptom	Layer isolated	Primary script
A	Not assigned / device absent	Assignment	`INTCFG-Discovery.ps1`
B	Assigned, not applied	Scope / Sync / CSP	`INTCFG-Validation.ps1`
C	Conflict / value flips	CSP apply (conflict)	`INTCFG-Report.ps1`
D	Filter excludes device	Scope (filter)	`INTCFG-Discovery.ps1`
E	Stuck sync / stale check-in	Sync	`INTCFG-Remediation.ps1`
F	Per-setting error code	CSP apply	`INTCFG-Validation.ps1`

> ***The repeatable way:*** *read the report → pick the branch → isolate the layer → apply that layer's typical fix → re-check the report. Repeat until* ***Device status = Succeeded*** *with no conflict. Overall model:* ***Assess → Configure → Pilot → Validate → Expand.***

5. Testing Guide — Intune Device Configuration & Policy Troubleshooting

Each test is a **pass/fail gate** that confirms a specific layer's fix actually worked. Run the test for the branch you resolved in the Diagnostic Playbook (document 3). A fix isn't "done" until its gate passes on both the affected device and — where relevant — a known-good device.

Test 1 — Assignment reaches the device (Branch A)

Do: Intune → **Devices** → **Configuration** → policy → **Device status**; and run `INTCFG-Discovery.ps1`. **Pass:** The affected device now appears in the policy's device list (no longer **Not applicable**), and Discovery lists the device's group under the policy's **Included** assignments with no overriding **Exclude**. **Fail:** Device still absent, or still in an Excluded group → return to Branch A.

Test 2 — Filter includes the device (Branch D)

Do: Devices → **Filters** → `INTCFG-Filter-Windows-Corporate` → **Preview devices (Evaluate)**. **Pass:** For an **Include**-mode filter the affected device **is** in the matched set (for **Exclude** mode it is **not**). **Fail:** Device on the wrong side of the filter → return to Branch D and correct the rule/mode.

Test 3 — Device has checked in (Branch E)

Do: Devices → **All devices** → device → **Last check-in**; and run `INTCFG-Validation.ps1` on the device. **Pass:** Last check-in is **recent** (after the assignment change) and Validation reports MDM enrollment **healthy**. **Fail:** Stale check-in or unhealthy enrollment → return to Branch E; run `INTCFG-Remediation.ps1 -WhatIf:$false` to force sync.

Test 4 — Setting applied on the device (Branch B)

Do: Policy → **Per-setting status** for the device; confirm the actual value on the device (registry/UI/`INTCFG-Validation.ps1`). **Pass:** The setting shows **Succeeded and** the observed value on the device matches the policy. **Fail:** Succeeded in Intune but wrong value → suspect conflict (Test 5); error code → Test 6.

Test 5 — No conflict on the setting (Branch C)

Do: Run `INTCFG-Report.ps1`; check the policy's **Per-setting status**. **Pass:** The setting appears in **exactly one** assigned policy (no overlap in the report) and shows no **Conflict** state. **Fail:** Setting still owned by more than one assigned policy → return to Branch C and remove the duplicate.

Test 6 — CSP error cleared (Branch F)

Do: Policy → **Per-setting status** → the previously failing setting; confirm on the device via `INTCFG-Validation.ps1` / the MDM report. **Pass:** The setting no longer reports an **error code**; it is **Succeeded** or a justified **Not applicable**. **Fail:** Same or new error code → recheck the code meaning in Branch F.

Test 7 — Known-good device still healthy (regression)

Do: Confirm the comparison device that was already **Succeeded** stays **Succeeded** after your change; run `INTCFG-Report.ps1` tenant-wide. **Pass:** No previously-working device regressed; no new conflicts introduced. **Fail:** A change broke another device/policy → review the assignment/filter/conflict you edited.

Pass/fail gate

Gate	Requirement
Isolation confirmed	The single failing layer is identified; the matching test above **passes** on the affected device
No regression	Test 7 passes — no previously-working device or policy was broken by the fix
Report clean	`INTCFG-Report.ps1` shows the target setting owned by one policy, device status **Succeeded**, no conflict

6. Troubleshooting Guide — Intune Device Configuration & Policy Troubleshooting

The **Diagnostic Playbook** (document 3) handles the common branches. This guide covers **deeper edge cases** the tree doesn't fully resolve, plus when and how to **escalate**. Use `INTCFG-Discovery.ps1` / `INTCFG-Report.ps1` for the tenant view, `INTCFG-Validation.ps1` on the device, and `INTCFG-Remediation.ps1` to force a sync.

A. Dynamic group membership lag

Symptom	Cause	Remediation
Device is in a dynamic group by rule, but the policy still doesn't target it	Dynamic membership **evaluation lag** (can take minutes to hours after a property changes)	Confirm the device matches the rule in Entra → Groups → membership; wait for processing, or temporarily add it to `INTCFG-Pilot-Devices` (static) to unblock; verify with `INTCFG-Discovery.ps1`
Device dropped out of scope unexpectedly	A device property changed and no longer matches the dynamic rule	Compare device Hardware properties to the rule; fix the rule or the property source

B. User-context vs device-context mismatch

Symptom	Cause	Remediation
Setting applies for some signed-in users but not at the device level (or vice-versa)	A user-scoped setting assigned to a **device** group, or a device-scoped setting assigned to a **user** group	Match assignment context to the setting: device settings → `INTCFG-Pilot-Devices`; user settings → `INTCFG-Pilot-Users`. Shared/multi-user devices amplify this — prefer device context there

C. "Succeeded" in Intune but wrong value on the device

- The report can show **Succeeded** while the effective value is wrong when a **conflict** exists

(last-writer wins) or a **local GPO / third-party agent** overwrites the same setting after MDM applies it. Run `INTCFG-Report.ps1` to rule out a policy conflict first; if clean, check for an on-device GPO or agent touching the same registry/CSP node.

D. Filter evaluates differently than expected

- Assignment filters evaluate against **device properties at check-in time**. A filter referencing a

property that is blank or not yet populated (e.g. `enrollmentProfileName` on a freshly enrolled device) can silently exclude the device. Use the filter's **Preview/Evaluate** and compare to the device's actual **Hardware** properties; avoid volatile properties in filter rules.

E. Settings-catalog vs template/ADMX behaviour

- The same outcome can be configured via a **settings-catalog** policy, a **template** profile, and

an **ADMX-ingested** policy — and these can target the **same CSP node** from different surfaces, producing a conflict that `INTCFG-Report.ps1` may only partially surface (it compares settings-catalog policies). If a value fights and no catalog overlap shows, check for a template or administrative-templates policy setting the same thing.

F. Sync succeeds but setting never appears

Symptom	Cause	Remediation
Last check-in is recent, no error, setting still absent	Setting is **Not applicable** to that OS **edition/build** (e.g. Enterprise-only on Pro)	Confirm applicability; target a supported edition. Not applicable is expected, not a failure
Recent sync, setting absent, no report line at all	Policy not actually assigned to this device (only appeared to be)	Re-confirm with `INTCFG-Discovery.ps1`; fix the assignment (Branch A)

G. Diagnostics quick reference

```
# On the device — confirm enrollment + last sync + CSP errors:
powershell -ExecutionPolicy Bypass -File .\INTCFG-Validation.ps1

# Force a check-in (default -WhatIf; add -WhatIf:$false to apply):
powershell -ExecutionPolicy Bypass -File .\INTCFG-Remediation.ps1 -WhatIf:$false

# MDM/CSP result events (error codes appear here):
Get-WinEvent -LogName 'Microsoft-Windows-DeviceManagement-Enterprise-Diagnostics-Provider/Admin' -MaxEvent
    Where-Object { $_.LevelDisplayName -in 'Error','Warning' }

# Common per-setting error codes:
#   0x87d1fde8  SyncML(500)  — conflict / internal (see Branch C)
#   0x87d101f4  SyncML(404)  — node not found / not supported on this OS build
#   0x82b00006  remediation failed — invalid value / missing prerequisite
```

H. When to escalate

Open a Microsoft support case when: a setting reports **Succeeded** yet is verifiably wrong on the device with **no** policy conflict and **no** on-device GPO/agent override; or a per-setting **error code** persists after the device is correctly targeted, in scope, freshly synced, and the setting is confirmed supported on that OS edition/build. Attach the device **MDM diagnostic report** (**Access work or school** → **Info** → **Create report**), the failing **error code**, the policy/setting name, and the outputs of `INTCFG-Discovery.ps1` and `INTCFG-Report.ps1` so the failing layer is already documented.

7. Best Practices — Intune Device Configuration & Policy Troubleshooting

Field-proven habits that prevent the failures the Diagnostic Playbook diagnoses. Adopt these and most "policy isn't applying" tickets never happen.

Naming (so scope is obvious at a glance)

- **Name policies for their scope and intent**, e.g. `INTCFG-Win11-Corp-BitLocker` — reading the name

tells you the platform, audience, and purpose without opening it.

- **Name groups by context**, splitting device from user: use `INTCFG-Pilot-Devices` and

`INTCFG-Pilot-Users` verbatim, and follow the same `-Devices` / `-Users` suffix in production. This single habit prevents the #1 Branch-A cause: device-context settings assigned to user groups.

- **Name filters by what they match**, e.g. `INTCFG-Filter-Windows-Corporate` — never leave a

filter called "New filter 1." A named filter's rule is knowable and reusable.

- **Reuse names verbatim** across policies, scripts, and documentation so diagnostics line up.

Filters (so scope is predictable)

- **Prefer one, well-understood filter** per platform over many overlapping ad-hoc filters.
- **Always Preview/Evaluate** a filter against real devices before assigning — confirm the intended

devices are on the correct side of Include/Exclude.

- **Filter on stable properties** (ownership, OS edition, enrollment profile) rather than volatile

ones; document what each filter is meant to match.

- **Remember Exclude beats Include** at both the group and filter level — check for accidental

excludes first when a device is missing.

Conflict-free design (so settings don't fight)

- **One setting, one owner.** Each setting should live in **exactly one** assigned policy. Overlap is

the root of every conflict — there is no reliable "priority wins" for configuration profiles.

- **Consolidate, don't stack.** Prefer fewer, broader well-scoped policies over many small policies

that touch the same settings.

- **Scope to non-overlapping groups** when two policies must coexist, so no device receives both.
- **Run** `INTCFG-Report.ps1` **before and after changes** to catch overlapping settings early.

Operating discipline (so diagnosis stays fast)

- **Pilot before production.** Assign to `INTCFG-Pilot-Devices` first, validate, then expand in waves.
- **Read the report first, always.** Device status + per-setting status tells you the failing layer before you touch anything.
- **Diagnose top-down** — assignment → filter → sync → CSP — matching the order settings travel.
- **Keep a known-good device** to compare against; it separates policy faults from device faults.
- **Force a sync deliberately**, don't wait blindly — but never re-push a policy as a "fix" before you've isolated the layer; blind re-pushing is what this toolkit replaces.
- **Document ownership** — record which policy owns which settings, and the group/filter scheme, so the next admin (and the scripts) can follow it.

8. Deliverables Summary — Intune Device Configuration & Policy Troubleshooting

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	How a policy travels assignment→scope→sync→CSP apply→report; the layered model behind the tree
2. Requirements Guide	Every prerequisite with Description / Why / Verify / Implement-if-missing
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Diagnostic Playbook	The symptom-driven decision tree — exact checks per branch and the layer each isolates
5. Testing Guide	Pass/fail gates confirming each layer's fix worked
6. Troubleshooting Guide	Deeper edge cases and escalation
7. Best Practices	Naming, filters, and conflict-free policy design

PowerShell scripts

Script	What it does
`INTCFG-Discovery.ps1`	Lists device-configuration/settings-catalog policies with their assignments and bound filters (read-only, Graph)
`INTCFG-Validation.ps1`	Per-device applied-state check: MDM enrollment health, last sync, and local CSP/policy status (read-only)
`INTCFG-Report.ps1`	Assignment + conflict coverage report — overlapping settings across assigned policies and their target groups (read-only, Graph)
`INTCFG-Remediation.ps1`	Safe re-sync / fix for a stuck device (forces MDM check-in); default `-WhatIf`
`INTCFG-Set-ScopeTargeting.ps1`	Creates/populates the pilot scope groups for clean targeting; default `-WhatIf`

What the buyer receives

- This diagnostic method and playbook as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP (Discovery/Validation/Report are read-only;

Remediation and Set-ScopeTargeting default to `-WhatIf`).

- A repeatable, layered root-cause process: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

The reason a policy isn't applying, isolated to **one layer** — assignment, filter, conflict, sync, or CSP — with a repeatable method and scripts you keep and reuse, instead of blind re-pushing.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

INTCFG-Discovery.ps1

Purpose: Enumerates Intune settings-catalog / device-configuration policies and, for each, lists the Included and Excluded Entra groups and any assignment filter (with its Include/Exclude mode). This is the assignment-layer picture the console shows one policy at a time. Read-only. Optionally filter by name with -NameLike (for example 'BitLocker'). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INTCFG-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\INTCFG-Discovery.ps1 -NameLike BitLocker Requires Microsoft Graph SDK and scopes DeviceManagementConfiguration.Read.All, Group.Read.All. EXPECTED RESULTS One block per policy: name, platform, Included/Excluded group IDs, and any bound filter + mode. "No configuration policies found" if none match. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$NameLike)

How to run: powershell -ExecutionPolicy Bypass -File .\INTCFG-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\INTCFG-Discovery.ps1 -NameLike BitLocker Requires Microsoft Graph SDK and scopes DeviceManagementConfiguration.Read.All, Group.Read.All. EXPECTED RESULTS One block per policy: name, platform, Included/Excluded group IDs, and any bound filter + mode. "No configuration policies found" if none match. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$NameLike)

INTCFG-Remediation.ps1

Purpose: For a device that is correctly assigned but has not received the policy (stuck sync), this triggers an MDM check-in so the assignment reaches the device, then reports the new sync result. It does NOT change any policy or setting. Defaults to -WhatIf: it shows what it would do and changes nothing until you re-run with -WhatIf:$false. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\INTCFG-Remediation.ps1 Apply the sync: powershell -ExecutionPolicy Bypass -File .\INTCFG-Remediation.ps1 -WhatIf:$false Run elevated on the affected device. No Graph scopes needed. EXPECTED RESULTS In -WhatIf: "WHATIF: would trigger MDM sync". Applied: "MDM sync triggered", then re-run INTCFG-Validation.ps1 to confirm a recent check-in and cleared setting status. .NOTES White-label — the repeatable way. Default -WhatIf. Confirm assignment BEFORE re-syncing. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([switch]$WhatIf = $true) # make -WhatIf the default while still honouring an explicit -WhatIf:$false if ($PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = [bool]$WhatIf } else { $WhatIfPreference = $true }

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\INTCFG-Remediation.ps1 Apply the sync: powershell -ExecutionPolicy Bypass -File .\INTCFG-Remediation.ps1 -WhatIf:$false Run elevated on the affected device. No Graph scopes needed. EXPECTED RESULTS In -WhatIf: "WHATIF: would trigger MDM sync". Applied: "MDM sync triggered", then re-run INTCFG-Validation.ps1 to confirm a recent check-in and cleared setting status. .NOTES White-label — the repeatable way. Default -WhatIf. Confirm assignment BEFORE re-syncing. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([switch]$WhatIf = $true) # make -WhatIf the default while still honouring an explicit -WhatIf:$false if ($PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = [bool]$WhatIf } else { $WhatIfPreference = $true }

INTCFG-Report.ps1

Purpose: Builds the tenant-wide conflict picture the console cannot show directly: it reads every settings-catalog / device-configuration policy, records each setting and the groups it targets, then reports any setting that appears in MORE THAN ONE assigned policy (overlap = conflict risk). Use it before and after changes to keep every setting owned by exactly one policy. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INTCFG-Report.ps1 Requires Microsoft Graph SDK and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A coverage summary (policies + assigned groups) and a CONFLICT RISK list of settings owned by more than one assigned policy. "No overlapping settings" if the estate is conflict-free. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param()

How to run: powershell -ExecutionPolicy Bypass -File .\INTCFG-Report.ps1 Requires Microsoft Graph SDK and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A coverage summary (policies + assigned groups) and a CONFLICT RISK list of settings owned by more than one assigned policy. "No overlapping settings" if the estate is conflict-free. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param()

INTCFG-Set-ScopeTargeting.ps1

Purpose: Builds the named pilot scope groups used to reproduce and fix targeting safely: INTCFG-Pilot-Devices and INTCFG-Pilot-Users. It creates each group if missing and adds the device/user object IDs you pass in. Use it to give a policy a small, known scope before touching production assignments. Defaults to -WhatIf: it previews every create/add and writes nothing until you re-run with -WhatIf:$false. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\INTCFG-Set-ScopeTargeting.ps1 -DeviceIds id1,id2 Apply: powershell -ExecutionPolicy Bypass -File .\INTCFG-Set-ScopeTargeting.ps1 -DeviceIds id1,id2 -WhatIf:$false Requires Microsoft Graph SDK and Group.ReadWrite.All (write scope only used when applying). EXPECTED RESULTS In -WhatIf: "WHATIF: would create/add ...". Applied: the groups exist and contain the members; then assign your policy to INTCFG-Pilot-Devices in Intune and validate. .NOTES White-label — the repeatable way. Default -WhatIf. Object IDs are Entra directory object IDs, not device names — use your-device-object-id / your-user-object-id from Entra. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([string[]]$DeviceIds = @(), [string[]]$UserIds = @(), [string]$DeviceGroupName = 'INTCFG-Pilot-Devices', [string]$UserGroupName = 'INTCFG-Pilot-Users', [switch]$WhatIf = $true) if ($PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = [bool]$WhatIf } else { $WhatIfPreference = $true }

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\INTCFG-Set-ScopeTargeting.ps1 -DeviceIds id1,id2 Apply: powershell -ExecutionPolicy Bypass -File .\INTCFG-Set-ScopeTargeting.ps1 -DeviceIds id1,id2 -WhatIf:$false Requires Microsoft Graph SDK and Group.ReadWrite.All (write scope only used when applying). EXPECTED RESULTS In -WhatIf: "WHATIF: would create/add ...". Applied: the groups exist and contain the members; then assign your policy to INTCFG-Pilot-Devices in Intune and validate. .NOTES White-label — the repeatable way. Default -WhatIf. Object IDs are Entra directory object IDs, not device names — use your-device-object-id / your-user-object-id from Entra. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([string[]]$DeviceIds = @(), [string[]]$UserIds = @(), [string]$DeviceGroupName = 'INTCFG-Pilot-Devices', [string]$UserGroupName = 'INTCFG-Pilot-Users', [switch]$WhatIf = $true) if ($PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = [bool]$WhatIf } else { $WhatIfPreference = $true }

INTCFG-Validation.ps1

Purpose: Run this ON an affected device to check the sync and CSP-apply layers locally: MDM enrollment health, the last successful sync time, and any recent MDM/CSP result events (including error codes) from the device's own logs. Read-only — it changes nothing on the device. HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\INTCFG-Validation.ps1 No Graph scopes needed — reads local registry and event logs only. EXPECTED RESULTS Enrollment health (healthy / not enrolled), last sync time, and a short list of recent CSP results with any error codes. "MDM enrollment not found" if the device is not Intune-managed. .NOTES White-label — the repeatable way. Read-only against the local device. #> [CmdletBinding()] param()

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\INTCFG-Validation.ps1 No Graph scopes needed — reads local registry and event logs only. EXPECTED RESULTS Enrollment health (healthy / not enrolled), last sync time, and a short list of recent CSP results with any error codes. "MDM enrollment not found" if the device is not Intune-managed. .NOTES White-label — the repeatable way. Read-only against the local device. #> [CmdletBinding()] param()